

Avance Fase 3 — Semana 2 (Formativa)

Grupo 4 · MCDI500

Este notebook parte del conjunto ya limpio y validado en la Fase 2

(`data/processed/ens_procesado.csv`). No se rehace la limpieza ni se agregan datos nuevos: el objetivo de esta fase es reorganizar el código que ya funciona, medir su eficiencia, y decidir si el proyecto justifica recursividad.

```
In [1]: import sys
import time
import pandas as pd
from pathlib import Path

SEMILLA = 2026

def encontrar_raiz_proyecto(marcador=".git"):
    """Sube por las carpetas padre hasta encontrar la raíz del repositorio."""
    actual = Path.cwd()
    for carpeta in [actual, *actual.parents]:
        if (carpeta / marcador).exists():
            return carpeta
    raise FileNotFoundError(f"No se encontro '{marcador}' en ningun directorio")

RAIZ = encontrar_raiz_proyecto()
print("Python:", sys.version.split()[0])
print("pandas:", pd.__version__)
print("Raiz del proyecto:", RAIZ)
```

Python: 3.13.7

pandas: 3.0.5

Raiz del proyecto: /Users/nyevenes/DevProjects/MagisterIA/proyecto-grupo4-mcdi500

```
In [2]: ARCHIVO = RAIZ / "data" / "processed" / "ens_procesado.csv"
df = pd.read_csv(ARCHIVO)

print(f"Archivo: {ARCHIVO}")
print(f"Conjunto cargado: {df.shape[0]} filas x {df.shape[1]} columnas")
df.head()
```

Archivo: /Users/nyevenes/DevProjects/MagisterIA/proyecto-grupo4-mcdi500/data/processed/ens_procesado.csv

Conjunto cargado: 5511 filas x 23 columnas

Out [2]:

	Edad	HTA	IMC	anos_estudio_MINSAL_1	GPAQ	as27	as28	Fexp
0	-1.256904	0	-1.031300	15.0	3	-0.433713	3.0	
1	-1.516452	0	-0.490394	14.0	3	NaN	NaN	
2	1.857664	1	-0.935179	15.0	1	1.339974	10.0	
3	-0.997357	0	-0.704314	16.0	3	NaN	NaN	
4	0.196561	0	0.604864	14.0	1	NaN	NaN	

5 rows × 23 columns

El conjunto antes de tocarlo

Antes de escribir cualquier función, revisamos brevemente el estado del conjunto que ya dejó lista la Fase 2. No se modifica nada aquí: es solo el punto de partida para las mediciones de esta fase.

In [3]:

```
print("Dimensiones:", df.shape)
print("\nTipos de datos:")
print(df.dtypes.value_counts())
print("\nValores nulos totales:", int(df.isnull().sum().sum()))
print("\nColumnas con nulos declarados (GPAQ imputada, as27/as28 con NaN genuino):")
print(df.isnull().sum()[df.isnull().sum() > 0])
```

Dimensiones: (5511, 23)

Tipos de datos:

int64 17

float64 6

Name: count, dtype: int64

Valores nulos totales: 1632

Columnas con nulos declarados (GPAQ imputada, as27/as28 con NaN genuino):

as27 816

as28 816

dtype: int64

3. Medición de eficiencia: búsqueda por posición

IdEncuesta se excluyó del conjunto procesado en la Fase 2, por no aportar valor analítico. Para esta medición trabajamos con el índice propio del DataFrame como identificador de cada persona, comparando dos formas de ubicar un registro: recorriendo el conjunto fila por fila frente a acceder directamente por posición.

```
In [4]: import sys
sys.path.append(str(RAIZ / "src"))

from medicion import buscar_recorriendo, buscar_por_indice, medir_tiempo, me

print("Funciones importadas correctamente desde src/medicion.py")

posicion_prueba = df.shape[0] - 1

r_a = buscar_recorriendo(df, posicion_prueba)
r_b = buscar_por_indice(df, posicion_prueba)

print("Encontrado por recorrido:", r_a is not None)
print("Encontrado por indice:", r_b is not None)
```

Funciones importadas correctamente desde src/medicion.py
 Encontrado por recorrido: True
 Encontrado por indice: True

3.1 Medición: tiempo de cada versión

Aplicamos las tres reglas para que la medición valga: repetir y conservar el mínimo, medir sobre el tamaño real del conjunto (5.511 filas), y comprobar que ambas versiones entregan el mismo resultado antes de comparar los tiempos.

```
In [5]: # posicion_prueba se define aqui mismo, sin depender de celdas anteriores
posicion_prueba = df.shape[0] - 1

tiempo_a, resultado_a = medir_tiempo(buscar_recorriendo, df, posicion_prueba)
tiempo_b, resultado_b = medir_tiempo(buscar_por_indice, df, posicion_prueba)

print(f"Recorriendo: {tiempo_a:.6f} s")
print(f"Por indice: {tiempo_b:.6f} s")
print(f"El acceso por indice es {tiempo_a / tiempo_b:.1f} veces mas rapido")

assert resultado_a.equals(resultado_b), "Las dos versiones no coinciden"
print("\nVerificado: ambas versiones entregan exactamente el mismo resultado")
```

Recorriendo: 0.030933 s
 Por indice: 0.000017 s
 El acceso por indice es 1776.1 veces mas rapido

Verificado: ambas versiones entregan exactamente el mismo resultado.

Por indice: 0.000017 s
 El acceso por indice es 1776.1 veces mas rapido

Verificado: ambas versiones entregan exactamente el mismo resultado.

3.2 Interpretación

El acceso por posición (`buscar_por_indice`) resultó **1.683 veces más rápido** que recorrer el conjunto fila por fila (`buscar_recorriendo`), midiendo sobre el peor caso posible (la última fila del conjunto).

La diferencia se explica por la naturaleza de cada operación: `buscar_recorriendo` tiene una complejidad temporal de **$O(n)$** , ya que en el peor caso revisa las 5.511 filas una por una antes de encontrar la buscada. `buscar_por_indice` , en cambio, es prácticamente **$O(1)$** : pandas accede directamente a la posición de memoria correspondiente, sin recorrer nada.

Esta diferencia se replica en el propio pipeline del proyecto: por ejemplo, en la Fase 2 se verificó la unicidad de `IdEncuesta` y se cruzaron los valores de `as27` y `as28` entre sí. Un enfoque que recorra el conjunto completo cada vez que se necesita ubicar un registro se vuelve costoso a medida que crece la muestra; indexar primero (como ya se hace de forma nativa en pandas con `.loc / .iloc`) es la opción adoptada para cualquier búsqueda repetida sobre el conjunto.

3.3 Medición de memoria

El descriptor pide analizar complejidad temporal **y** espacial. El acceso por índice suele ganar en tiempo, pero implica mantener una estructura adicional en memoria. Medimos ambos aspectos con `tracemalloc` , siguiendo la sugerencia de la retroalimentación recibida

```
In [6]: memoria_recorrido, _ = medir_memoria(buscar_recorriendo, df, posicion_prueba)
        memoria_indice, _ = medir_memoria(buscar_por_indice, df, posicion_prueba)

        print(f"Memoria maxima - recorrido: {memoria_recorrido / 1024:.2f} KB")
        print(f"Memoria maxima - por indice: {memoria_indice / 1024:.2f} KB")
```

```
Memoria maxima - recorrido: 996.35 KB
Memoria maxima - por indice: 2.28 KB
```

3.3.1 Interpretación de la memoria

Contrario a lo esperado en el caso general (donde construir un índice suele costar memoria adicional), en esta comparación el acceso por índice también resultó más eficiente en memoria: 2,28 KB frente a 996,24 KB del recorrido.

La explicación está en cómo opera cada función: `buscar_recorriendo` usa `iterrows()` , que construye una nueva `Series` de pandas en cada iteración del bucle -consumiendo memoria temporal en cada paso, aunque se descarte después-, mientras que `buscar_por_indice` con `.iloc` accede directamente a la posición en memoria ya existente del DataFrame, sin crear estructuras intermedias.

Esto no contradice el principio general (que construir un índice formal, como con `.set_index()`, sí tiene un costo de memoria), sino que muestra que **el costo real depende de la implementación específica**, no solo del enfoque conceptual. Por eso es imprescindible medir en el caso concreto, y no asumir el resultado de memoria.

3.3.2 Verificación: ¿el resultado es consistente?

Antes de confiar en una sola medición, la repetimos varias veces para confirmar que la diferencia de memoria no es casualidad.

```
In [7]: resultados_recorrido = []
resultados_indice = []

for _ in range(5):
    mem_r, _ = medir_memoria(buscar_recorriendo, df, posicion_prueba)
    mem_i, _ = medir_memoria(buscar_por_indice, df, posicion_prueba)
    resultados_recorrido.append(mem_r)
    resultados_indice.append(mem_i)

print("Memoria recorrido (5 mediciones, en KB):", [round(m/1024, 2) for m in resultados_recorrido])
print("Memoria indice      (5 mediciones, en KB):", [round(m/1024, 2) for m in resultados_indice])

print(f"\nRecorrido - minimo: {min(resultados_recorrido)/1024:.2f} KB, maximo: {max(resultados_recorrido)/1024:.2f} KB")
print(f"Indice      - minimo: {min(resultados_indice)/1024:.2f} KB, maximo: {max(resultados_indice)/1024:.2f} KB")
```

Memoria recorrido (5 mediciones, en KB): [996.31, 994.61, 994.39, 994.39, 994.39]

Memoria indice (5 mediciones, en KB): [2.09, 2.09, 2.09, 2.09, 2.09]

Recorrido - minimo: 994.39 KB, maximo: 996.31 KB

Indice - minimo: 2.09 KB, maximo: 2.09 KB

3.3.3 Conclusión de la verificación

Las 5 repeticiones confirman que el resultado es estable: la memoria usada por `buscar_recorriendo` se mantiene siempre entre 994 y 996 KB, y la de `buscar_por_indice` es constante en 2,09 KB. La diferencia (~475 veces menos memoria con el índice) no es un dato aislado, sino un patrón reproducible atribuible a la diferencia estructural entre `iterrows()` y `.iloc` explicada arriba.

3.4 Punto de equilibrio: ¿cuándo conviene cada enfoque?

`buscar_por_indice` no tiene un costo de "construcción" separado (no se crea una estructura adicional, `.iloc` opera directo sobre el DataFrame ya existente). Por eso, a diferencia del caso con `.set_index()`, el índice es más rápido **desde la primera**

búsqueda: no hay un número de repeticiones a partir del cual empiece a convenir, conviene siempre.

Esto se debe a que `buscar_recorriendo` tiene complejidad $O(n)$ por cada búsqueda individual, mientras que `buscar_por_indice` es $O(1)$ por búsqueda, sin ningún costo inicial que amortizar.

```
In [8]: # Se compara el costo ACUMULADO de hacer N búsquedas con cada enfoque
for n_búsquedas in [1, 10, 100]:
    tiempo_recorrido_total = 0
    tiempo_indice_total = 0
    for _ in range(n_búsquedas):
        t_r, _ = medir_tiempo(buscar_recorriendo, df, posicion_prueba, repeti
        t_i, _ = medir_tiempo(buscar_por_indice, df, posicion_prueba, repeti
        tiempo_recorrido_total += t_r
        tiempo_indice_total += t_i

    print(f"Con {n_búsquedas:>3} búsquedas -> recorrido: {tiempo_recorrido_t
          f"índice: {tiempo_indice_total:.6f}s, "
          f"razón: {tiempo_recorrido_total/tiempo_indice_total:.1f}x")
```

```
Con 1 búsquedas -> recorrido: 0.0338s, índice: 0.000080s, razón: 422.1x
Con 10 búsquedas -> recorrido: 0.3303s, índice: 0.000734s, razón: 449.8x
Con 100 búsquedas -> recorrido: 3.1547s, índice: 0.006922s, razón: 455.7x
```

3.4.1 Interpretación del punto de equilibrio

Los resultados confirman que **no existe un número de búsquedas a partir del cual el índice empiece a convenir: conviene desde la primera**, y su ventaja se amplifica mientras más búsquedas se realicen:

Búsquedas	Recorrido	Índice	Ventaja del índice
1	0,0216 s	0,000064 s	338.1x
10	0,2061 s	0,000496 s	415.3x
100	1,9387 s	0,003732 s	519.5x

Esto se explica porque `buscar_recorriendo` tiene un costo de **$O(n)$ por cada búsqueda individual** (recorre hasta 5.511 filas cada vez), por lo que el tiempo total crece linealmente con el número de búsquedas. `buscar_por_indice`, en cambio, tiene un costo casi constante por búsqueda (**$O(1)$**), por lo que su tiempo total crece mucho más lento.

Decisión adoptada: dado que el proyecto necesita ubicar registros repetidamente durante la validación (por ejemplo, al verificar cruces entre `as27` y `as28`, o al comprobar unicidad de identificadores), el acceso por índice es la opción adoptada, sin ninguna condición que lo desaconseje.

4. ¿Necesita este proyecto recursividad?

Antes de forzar una recursión artificial, evaluamos si el pipeline de la Fase 2 tiene un problema que la justifique.

```
In [9]: # Revision de los pasos del pipeline de Fase 2:
pasos_pipeline = [
    "seleccionar_variables_ens",
    "filtrar_ponderador_valido",
    "explorar_dataframe",
    "revisar_codigos_especiales",
    "marcar_codigos_no_respuesta",
    "imputar_nulos_numericos / imputar_nulos_categoricos",
    "codificar_one_hot",
    "escalar_caracteristicas",
    "validar_dataset",
]

print(f"El pipeline tiene {len(pasos_pipeline)} pasos, en una secuencia FIJA")
for i, paso in enumerate(pasos_pipeline, 1):
    print(f" {i}. {paso}")
```

El pipeline tiene 9 pasos, en una secuencia FIJA y conocida:

1. seleccionar_variables_ens
2. filtrar_ponderador_valido
3. explorar_dataframe
4. revisar_codigos_especiales
5. marcar_codigos_no_respuesta
6. imputar_nulos_numericos / imputar_nulos_categoricos
7. codificar_one_hot
8. escalar_caracteristicas
9. validar_dataset

4.1 Conclusión: no se justifica recursión en el pipeline principal

El pipeline principal tiene una secuencia fija y conocida de nueve pasos, sin niveles de profundidad variable ni una estructura autosimilar. Un enfoque iterativo (aplicar cada función en orden) es preferible: es más simple, más legible, y no arriesga agotar la pila de llamadas de Python.

La única función recursiva presente en el proyecto es `aplanar()`, utilizada en la Fase 1 para convertir el diccionario anidado de metadatos del proyecto en pares clave-valor. Esa función sí se justifica, porque la profundidad de anidamiento del diccionario no se conoce de antemano al escribir el código: podría tener uno, dos o más niveles, y la recursión se adapta a cualquiera de ellos sin necesidad de escribir un bucle distinto para cada caso.